An explicit Diophantine specialization for Con(ZFC)

A small factored polynomial, a nine-equation alternative, and a formal-library route
22 October 2026

Main result. The construction below specifies one ordinary integer polynomial in **18 positive unknowns**, of exact degree **187**, evaluated by an **82-operation** straight-line program. Its lack of positive integer solutions is provably equivalent, by the mathematical reductions below formalizable in ZFC, to Con(ZFC). Replacing each unknown by $1 + x_i$ gives a polynomial on $\mathbb{N}^{18}$ with the same property.

What "explicit" means here. All coefficients are uniquely specified by a finite, deterministic algorithm starting from the bundled literal K215 transition table. There is no unspecified universal-machine index or existential choice of coefficients. The enormous coefficients have *not* been expanded into decimal or binary numerals, and the full finite window alphabet has not been enumerated. This is an explicit coefficient-recipe construction, not a materialized coefficient list.

What "small" means here. The 82 count charges binary additions, subtractions and multiplications in the saved shared-expression schedule, with fixed integer coefficients supplied for free, following the repository's convention. It does not charge coefficient bit lengths or coefficient generation. No global minimality is claimed. A nine-equation system in 20 positive unknowns offers lower degree and a shorter comparison certificate; an alternative based on the existing Lean tag-system development offers a different validation path.

Pin and status. All construction inputs are pinned to ProveIt commit

`cdd582bec13f908df3711fa9e05d1ddac368a5db`.

This report supplies a conventional mathematical construction using the repository's native universality proofs. That native construction is not fully Lean-formalized. The new adapters and complete Diophantine endpoint have not been kernel-checked here. Existing Lean/Rocq endpoint proofs and finite regression tests are kept distinct from that missing check.

1. The actual equation

Fix the six integers $\beta_0, K, t_0, u_0, M_C, M_F$ specified in Sections 3–5. They are constants, not unknowns. The 18 strictly positive unknowns, in order, are

$$J, F, \alpha, z, f, h, i, T, s, w, \tau, \eta, \zeta, y, Z, \delta, \rho, \sigma.$$

Define the following *polynomial abbreviations*, not extra existential variables:

$$\begin{aligned}
q &= \beta_0 J + 1, & X &= wq, & Y &= sq^3, & E &= XY,\\
k &= \eta + \zeta, & c &= kY + \eta, & a &= E + Y, & H &= 4a + 3,\\
\Delta &= a^2 + H, & D &= X + ac + (\rho + \sigma)H, & & &\\
C &= q - F - Z - \alpha - t_0, & W &= C - Z, & \kappa &= u_0 + \delta\Delta, &\\
\mu &= W + a\kappa + \rho H, & & & & &\\
R_* &= (q^2 - qF - Z)(q^2 - 1) + (M_C + qM_F)J, & & & & &\\
L_* &= E(kY), & V &= c(Tf - 1) - R_* f^2, & A_* &= (i\Delta c^2)^2. &
\end{aligned}$$

Set

$$N_1 = \tau^2 - L_*(L_* + k), \qquad N_2 = D^2 - \Delta c^2,$$
$$N_3 = \mu^2 - \Delta\kappa^2, \qquad N_4 = A_*(V^2 - y^2) + y^2,$$
$$N_5 = k - hE - R_*, \qquad N_6 = (K + w)C + (q - F) - z(q - 1),$$
$$N_7 = \Delta f^2 - A_*.$$

The requested equation is

$$\boxed{\mathcal{P} = N_1N_2N_3N_4N_5N_6N_7 - \Delta = 0.} \tag{1}$$

All subtraction is ordinary integer subtraction. No unknown occurs in an exponent. In particular, this is not an exponential Diophantine equation. The many polynomial abbreviations must not be mistaken for extra variables or equations.

Theorem 1 (Coefficient-defined ZFC instance)**.** *For the constants specified below,*

$$\mathrm{ZFC} \vdash \big(\neg\exists \mathbf{v} \in \mathbb{N}^{18}_{>0}\, \mathcal{P}(\mathbf{v}) = 0\big) \iff \mathrm{Con(ZFC)}.$$

Equivalently, $\mathcal{P}(x_1+1, \ldots, x_{18}+1) = 0$ *has no solution in* $\mathbb{N}^{18}$ *exactly when ZFC is consistent, with the equivalence provable in ZFC.*

Here Con(ZFC) means that no finite proof of contradiction exists in the fixed ordinary first-order ZFC calculus used by the pinned verification endpoint. Section 7 explains the proof and its formalization boundary. The theorem asserts an equivalence, not an unconditional proof of either consistency or nonexistence of solutions.

2. Why 82 operations, and why degree 187?

The repository's complete native polynomial F_{84} uses 18 positive witnesses and one varying ordinary input x. The compiled ordinary-input language contains every positive x exactly when K215 halts. We need only the fixed instance $x = 1$.

Two rows of its exact schedule become fixed numerals:

$$\texttt{scaled_t} = (2d)x = 2d, \qquad \texttt{odd_index} = \texttt{scaled_t} + b = 2d + b.$$

Removing their one multiplication and one addition changes $47M + 37A$ to $46M + 36A = 82$. The saved JSON gives every remaining instruction. A literal conversion to nonnegative inputs adds 18 input additions if those gates are priced. This is a fixed-input specialization, *not* an improvement of the universal ordinary-input 84-operation bound.

The exact-degree proof for the parent has a degree-187 monomial, independent of x, with coefficient $-32(B-1)^{112}$. Since the compiled $B > 1$, this coefficient remains nonzero after $x = 1$. Specialization cannot increase degree, so the resulting degree is exactly 187. Translation $v_i = x_i + 1$ preserves the degree.

The parent's soundness proof does not infer that seven factors are units merely because their product is Δ. It proves the full identity $F_{84} = \Delta F_{85}$ and establishes

$$\Delta = (a + 1)(a + 3) > 0$$

from the supplied positive coordinates *before* cancelling it. This is why arbitrary signed inputs are not allowed in (1). Off that domain, $\Delta = 0$ can produce spurious zeros.

3. The fixed program: no unspecified proof-search index

Use `candidate-k215/k215.tm`, bundled byte-for-byte. It has 215 working states, binary alphabet, initial state 0, blank symbol 0 and halt target 215. Its SHA-256 is

`fdbcef0825886035c3091c8537a69535bc96f9e265f0ae45863543d40ae70090`.

The pinned K215 endpoint relates blank halting to a finite ZFC refutation. Its logical composition includes the ZF/ZFC equiconsistency bridge; this is not silently substituted for a ZFC endpoint.

For the native compiler, define a one-sided machine M with 437 states:

$$q_0, q_1, E_0, E_1, \text{Return}, \text{Loop}, \text{Halt}, \quad (q, \text{upper}), (q, \text{lower}) \quad (0 \le q < 215).$$

State numbers are $0, \dots, 6$ for the first seven, then $7 + 2q + \text{side}$. Its eight tape symbols encode two bits and an origin flag. Start with raw code $u + 2\ell + 4o$ and transpose codes 2 and 4. Thus blank is 0, unary input symbol is 1 and a flagged origin with zero bits is 2.

On an input of $y + 2$ ones at positions $-(y + 1), \dots, 0$, q_0 writes 2 and stays in q_1. Then q_1 moves left into E_0. Each erased 1 toggles the two E states. At the first blank, E_1 returns right to the origin; E_0 enters the nonhalting loop. Return moves through zeros to symbol 2 and starts K215's simulation. All other preparation cases loop. This accepts the preparation exactly for even y, because $y + 1$ ones lie left of the origin.

At a physical position $p \le 0$, the upper track represents virtual cell $-p$, and the lower track virtual cell $p - 1$. A right move on upper or left move on lower is a physical left move. The opposite moves go physically right, except at the flagged origin, where they stay and switch tracks. Write only the selected track; preserve the origin flag. Copy the next K215 state, or enter Halt when its target is 215.

Induction on steps proves exact tape/head/state correspondence. Consequently, for positive y,

$$M \text{ halts on } y + 2 \text{ ones} \quad \Longleftrightarrow \quad y \text{ is even and K215 halts on blank.}$$

This is exactly the native compiler's S_{even} convention. The final ordinary input $x = 1$ denotes raw distance $y = 2$, with *four* initial ones, not three.

4. The finite window alphabet

There are precisely 3510 tile types: a vertical boundary V, a horizontal boundary H, the $8 \cdot 438$ ordinary symbol/head tiles (head absent or one of 437 states), and four initialization-phase tiles L, I, Q, R. Their exact order and the full radius-one predicate are defined in `native/k215_native_definition.py`.

The predicate propagates horizontal boundaries only between nonvertical-boundary cells, enforces the initialization phase path $L^+I^+QR^+$, applies the three-neighbor transition of M, forbids head escape and permits a final-row head only in Halt. Each window is a row-major nine-tuple. The Start and End markers below use (a, q, p) for symbol, optional head and optional phase; a dash denotes absence:

$$\begin{array}{ccc|ccc} H & H & H & H & H & H \\ (1,-,I) & (1,-,I) & (1,q_0,Q) & (0,-,L) & (1,-,I) & (1,-,I) \\ (1,-,-) & (1,-,-) & (2,q_1,-) & (0,-,-) & (1,-,-) & (1,-,-) \end{array}.$$

List Start first, End second, then all other allowed windows in lexicographic tile-ID order. This defines a unique finite list $\mathcal{W}$, of length $k_0 \le 3510^9$. The selector order is essential: the 75/76 compiler uses Start 0 and End 1, unlike the earlier 81 arithmetic presentation. The overlapping-window and marker-bijection proofs handle arbitrary valid helical configurations, not only chosen sample histories.

5. All six coefficients, as fixed finite sums

The following recipe makes the constants in (1) unambiguous without writing their digits. It is implemented independently in the bundled standard-library Python file. Every enumeration and exponent in this section is fixed before choosing the Diophantine unknowns.

Write $n = 3510$, $k_0 = |\mathcal{W}|$, and $A_0 = 2^{\max(2,\mathrm{bitlength}(k_0+1))}$. For $0 \le r, c < 3$ and $0 \le t < n$, set

$$e(r,c,t) = k_0 + 3n + (2-r)3n + (2-c)n + t, \qquad j(r,c,t) = 1 + (3r+c)n + t.$$

Selector positions are $0, \dots, k_0 - 1$. Give each selector coefficient 1. Give position $e(r,c,t)$ coefficient $A_0^{j(r,c,t)}$, and add that same coefficient to each selector whose window has tile t at (r,c). Add four further clause coefficients $A_0^{9n+1}, \dots, A_0^{9n+4}$ to each selector.

Start with $\mu_0 = A_0 - 2 + \sum_{j=1}^{9n+4} A_0^j$ and $m = 2\,\mathrm{popcount}(\mu_0) + 12n + 2$. Append consecutive zero-expression mask-one clauses until $m + 1 \ge k_0 + 9n + 5$, adding the next power of A_0 to μ_0 and 2 to m each time. Put $m + 1 - k_0 - 9n - 4$ dummy positions consecutively from $k_0 + 12n$; this is at least one position.

Let E_0 be the largest selector/copy/dummy position, $M_0 = E_0 + 3n + 1$, and place four anchors at $M_0, 3M_0, 9M_0, 27M_0$, with the four extra clause coefficients. Let $\mathcal{E}$ be all these positions and write c_e for the assigned coefficients (dummy coefficients zero). Set

$$\begin{aligned} E_{\max} &= 27M_0, & H_0 &= E_{\max} + 24M_0 + 3n + 1, \\ T_1 &= H_0 + 2E_{\max} + n + 1, & T_2 &= T_1 + 2E_{\max} + 1, \\ g &= T_2 + E_{\max} + 1, & \ell &= \min\{5^j : 5^j \ge g + E_{\max} + 1\}. \end{aligned}$$

Define

$$D_0(R) = R^{3n} + R^{H_0+n} + R^{8M_0} + R^{24M_0} + \sum_e c_e (R^{T_1-e} + R^{T_2-e}).$$

Let $\epsilon = 1$ if $2D_0(2) - 2^{H_0} \equiv 0 \pmod 5$, otherwise 0; put $D_C(R) = D_0(R) + \epsilon R^g$ and $D_R(R) = R^{H_0}$. Choose

$$b = \min\left\{5^j : 5^j \ge \mathrm{bitlength}(Q_0 - 1)\right\}, \quad Q_0 = \max\{4(|\mathcal{E}| + 2)(2\sum_e c_e + 6) + 8,\ 2\mu_0 + 4,\ 16\}.$$

Now $R = 2^b$, $d = b\ell$, $B = 2^d$. This is the strengthened 75 radix choice; changing it to the older 76 choice is not justified. If e_* is the first dummy position, set

$$\begin{aligned} M_C &= B - 1 - \sum_{e \in \mathcal{E},\ e \ne 1} R^e - 2R^{e_*}, \\ M_{F,0} &= \mu_0(R^{T_1} + R^{T_2}) + \sum_{r<2, c<3, t<n} R^{e(r,c,t)} + \sum_{r<3, c<2, t<n} R^{H_0 + e(r,c,t)} + R^{9M_0} + R^{27M_0}. \end{aligned}$$

Finally, the six coefficients of the equation are

$$\boxed{\beta_0 = B - 1, \quad K = D_C(R) + BD_R(R), \quad t_0 = 2d, \quad u_0 = 2d + b, \quad M_C, \quad M_F = M_{F,0} + 4 + B - 1.}$$

The source coefficient M_F includes $B - 1$. The mask range bound applies to $M_{F,0} + 4$, not to M_F. This shift is not optional.

6. Alternatives and domain conversions

Nine equations, 20 positive unknowns. The accompanying `native/k215_native72_system.json` gives nine complete polynomial comparisons in 20 positive unknowns, with the *same* six fixed coefficients. It specializes the repository's signed-intermediate, positive-input 74-operation construction at $x = 1$. Only computed intermediates may be signed; all supplied witnesses remain positive.

The resulting comparison schedule costs $39M + 33A = 72$. Subtracting each pair, squaring all nine residuals and adding them costs another $9 + 9 + 8 = 26$ operations. Thus one sum-of-squares polynomial costs **98 operations** and has **exact degree 84**. This is a useful lower-degree alternative to (1), not a claim that the 72-operation comparison system is already a one-polynomial evaluator.

All-integer unknowns. To use unrestricted integers, replace every positive coordinate in (1) by

$$v_j = 1 + a_j^2 + b_j^2 + c_j^2 + d_j^2.$$

Lagrange's four-square theorem proves that this reaches exactly the positive integers. The result is one polynomial in **72 integer unknowns**, of exact degree **374**. The 82-operation count does not include these conversion gates. Simply allowing signed witnesses in the original equation is wrong.

A quartic alternative. The arithmetic circuit also yields a low-degree system. Keep the first 81 of the 82 gates as equations, and replace the final subtraction/output by the equation $\texttt{seven_units} = \Delta$. Give each intermediate an arbitrary integer value. For a natural-variable system, represent each signed intermediate as $z_i^+ - z_i^-$ and replace each positive input by $1 + x_j$. This gives **82 quadratic equations in 180 natural unknowns**; their sum of squares is one polynomial of degree at most four. Both directions follow by topological evaluation of the circuit. This trades many witnesses for low degree, and introduces no computational-history bound.

An alternative with a formalized compiler foundation. The archive also supplies the independently explicit K215-to-TM0 adapter and a complete 131,144-slot three-counter program. The binary alphabet is indexed explicitly and both input bits are encoded as blank, so the counter compiler's input 1 represents a blank tape. The adapter has 240 states: 215 original states plus 25 shared move helpers for the write-changing transitions. One source step becomes one or two TM0 steps.

The existing Lean development compiles this kind of three-counter program to a 30-tag system, then to a binary tag system, then to `Sys91`. Its 18 equations have 29 positive unknowns. Six positive definitional substitutions reduce these to 12 equations in 23 positive unknowns. A further parameterization of the head-geometry equation gives **11 equations in 22 natural unknowns**, and a single sum-of-squares polynomial of degree at most 348. The complete equations, finite production tables and closed finite-sum coefficient recipe are in `tag/tag_alternative.txt`.

This alternative does not invoke an existential DPRM index. Its finite program and coefficient algorithms are explicit. Its general arithmetic and simulation theorems are already present in Lean, but the newly supplied K215 adapter, fixed indexing and composite endpoint were *not* freshly compiled here. It is therefore a stronger starting point for formal integration, not a completed new formal certificate.

7. Proof of the consistency equivalence

Let $\mathrm{Ref}_{\mathrm{ZFC}}$ denote the existence of a finite contradiction proof in the pinned calculus. The construction uses the following implications in both directions:

$$\begin{aligned}\exists \mathbf{v} > 0\ \mathcal{P}(\mathbf{v}) = 0 &\iff \text{the compiled native instance accepts ordinary input 1}\\ &\iff M \text{ halts on the four-one input}\\ &\iff \text{K215 halts from blank}\\ &\iff \mathrm{Ref}_{\mathrm{ZFC}}.\end{aligned}$$

The first equivalence is the complete native positive-zero theorem, not a generic existence invocation of DPRM. Its hypotheses are discharged by the exact finite compiler: the two masks, radix margin, synchronization anchors, ordinary-input loader, positive ratios and strong auxiliary Pell condition are all retained. The F_{84} scaling step and the fixed-input substitution preserve the whole positive zero set.

The second equivalence is the marked helical compiler theorem. Start/End numbering and doubled input are fixed explicitly. The third is the folded-tape simulation invariant of Section 3. For the fourth, the K215 proof composes enumerator soundness/completeness, program simulation and its logical calibration, including the ZF-to-ZFC refutation bridge. Negating the final equivalence yields the theorem.

Why this is a ZFC proof claim rather than an appeal to Lean's soundness. The finite machine, finite alphabet, finite window set and finite sums defining coefficients are set-sized objects definable in ZFC. Their total construction does not test whether K215 halts. Machine simulation is ordinary induction on finite runs. The native arithmetic proof uses integer polynomial identities, finite packing, elementary number theory and Pell equations; these arguments are formalizable in ZFC. The logical calibration uses ordinary first-order syntax, completeness and the set-model/equiconsistency constructions described in the K215 development, also within ZFC. Thus the conventional mathematical proof can be formalized there.

The pinned repository expressly distinguishes its external Lean/Rocq equivalence from a derivation encoded in ZFC's own proof syntax. We do not claim to have produced such a proof-code object, nor that ZFC proves Lean soundness, nor that the new composed theorem has been checked by a proof assistant. The new theorem is supplied at the mathematical-construction level, with all reusable ingredients and integration gaps identified.

Independence qualification. If ZFC is consistent, Gödel's second incompleteness theorem prevents it from proving this no-solution statement, since that would prove its own consistency. Mere consistency alone does not justify the stronger assertion that neither side is provable. No unconditional consistency or nonhalting claim is being made.

8. What was actually checked

The construction includes exact source pins, explicit finite tables and independently written standard-library checks. The normalized machine has $437 \cdot 8 = 3496$ transitions. Tests covered preparation on inputs 1–12, both marked windows and 12,000 simulated K215 steps. The independent TM0 path checked 1,720 local adapter cases, 448 counter macro cases, eight input loaders and 11,760 TM0 simulation cases. These are regression tests; the general assertions rely on the stated invariants and compiler theorems.

The 82-row specialization was checked by exact instruction correspondence to the parent source. A separately written factored evaluator agrees with it on 120 integer/rational assignments. Such assignments test algebra only: arbitrary diagnostic coefficients are not claimed to encode valid programs. Small native-layout fixtures agree with the pinned compiler receipt. Full window enumeration, full coefficient expansion and a complete accepting Pell witness were not attempted. No proof assistant or received Python program was executed for this report.

9. Reproducibility, comparison and next formal step

The archive's primary artifact is the complete shared-expression schedule together with `native/k215_native_definition.py` and the pinned `k215.tm`. The six constants are the unique return values of its explicitly defined `k215_coefficients` function. Its default command runs only bounded regression checks; it intentionally does not begin the infeasible full enumeration. The coefficient definition remains a terminating finite algorithm independent of all unknowns and of consistency.

The formal-integration alternative contains a fully materialized three-counter instruction list and its generator, not merely a program name. The tag coefficients are given by exact finite sums over the fixed production table, with closed formulas for the initial word; neither astronomical binary words nor their contents need to be expanded to specify them. The alternative's newly supplied variable eliminations are elementary identities, with positivity checked in both directions.

Construction	Unknowns	Equations	Degree	Arithmetic
Native main, positive	18	1	187	82
Native comparison, positive	20	9	≤ 42	72
Native SOS, positive	20	1	84	98
Main, unrestricted integers	72	1	374	not repriced
Circuit lift, natural	180	82	≤ 2	not repriced
Circuit SOS, natural	180	1	≤ 4	not repriced
Tag alternative, natural	22	11	≤ 174	not repriced
Tag SOS, natural	22	1	≤ 348	not repriced

Counts are not comparable unless the domain, coefficient representation and circuit-sharing convention are retained. In particular, small operation count is not small bit-description length. The literal coefficient sizes are huge, and no reasonable claim of an absolute "simplest" Diophantine representation follows.

Recommended endpoint. Use (1) when the target is the smallest stated single-polynomial arithmetic count and fewest witnesses in this packet. Use the nine-equation version for lower-degree arithmetic and a more conventional comparison certificate. For a new machine-checked repository theorem, start with the tag alternative: connect the explicit indexed TM0 adapter to the existing `TMCounter.accepts_iff`, instantiate `tag_iff91`, and prove the short eliminations. This separates a tractable formal bridge from re-formalizing the complete native optimization chain.

Pinned sources and attribution

All ProveIt references use revision `cdd582bec13f908df3711fa9e05d1ddac368a5db`. Full paths, hashes and scope notes are in the archive. Principal sources are:

1. K215 verification, together with `KEndpoint215.lean` and `KEndpoint.lean`.
2. The complete native 75 proof and its linked modified helical compiler, kernel and input-loader proofs.
3. The complete 84-operation polynomial and the 74-operation comparison construction.
4. The constructive Lean files `TuringCounter100`, `TuringSim100`, `CounterIff100`, `TagGoedel91`, `TagRound91`, `TagBinary91`, `TagUniversal91`, `System91`, `Kernel3` and `TagEquiv91`.

The existing K215 and universal Diophantine constructions are prior repository work, drawing on the attributed Jones/Matiyasevich, register-machine and busy-beaver developments. The present contribution is their explicit fixed consistency instance, the complete deterministic coefficient specification, fixed-input arithmetic specialization, explicit adapters and the alternative eliminations. It is not a new universal-polynomial theorem or a proof of global minimality.